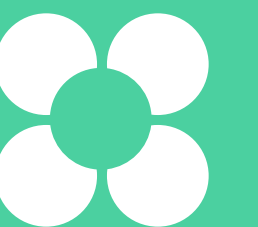
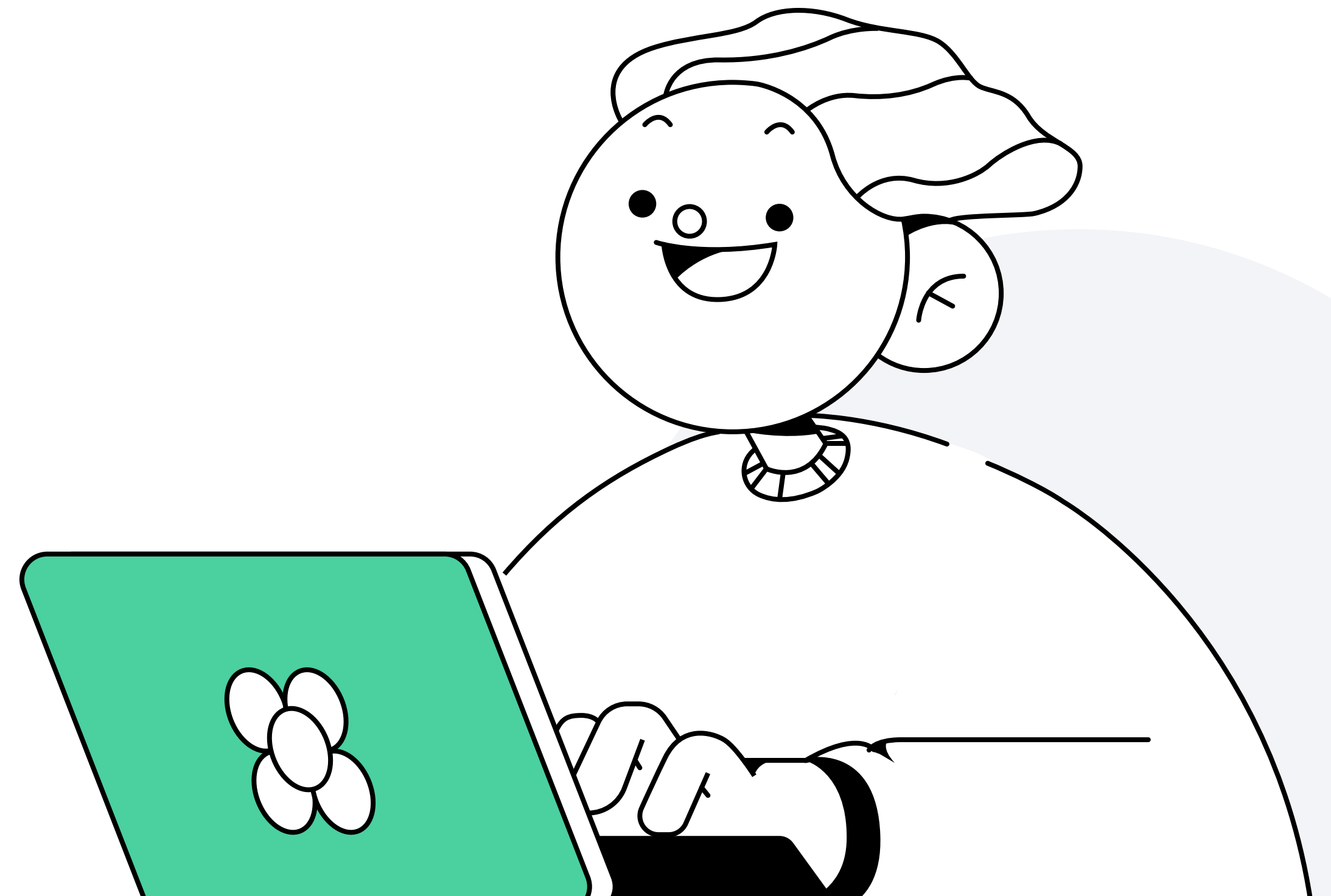


Docker: основы работы



План занятия

- 1 Введение в Docker
- 2 Работа с Docker



Введение в Docker

Тема занятия

- 1 Основные понятия
- 2 Инструкции Docker



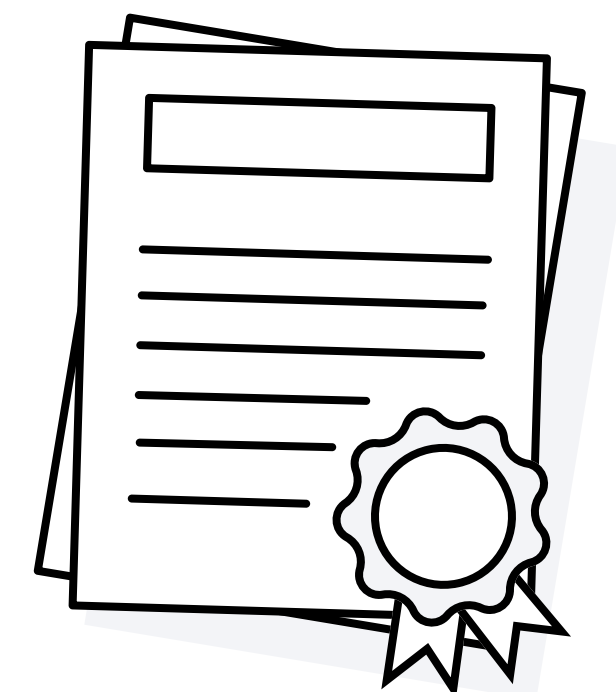
Docker

Docker — это программное обеспечение для развёртывания и управления переносимой среды, в которой может работать ваше приложение

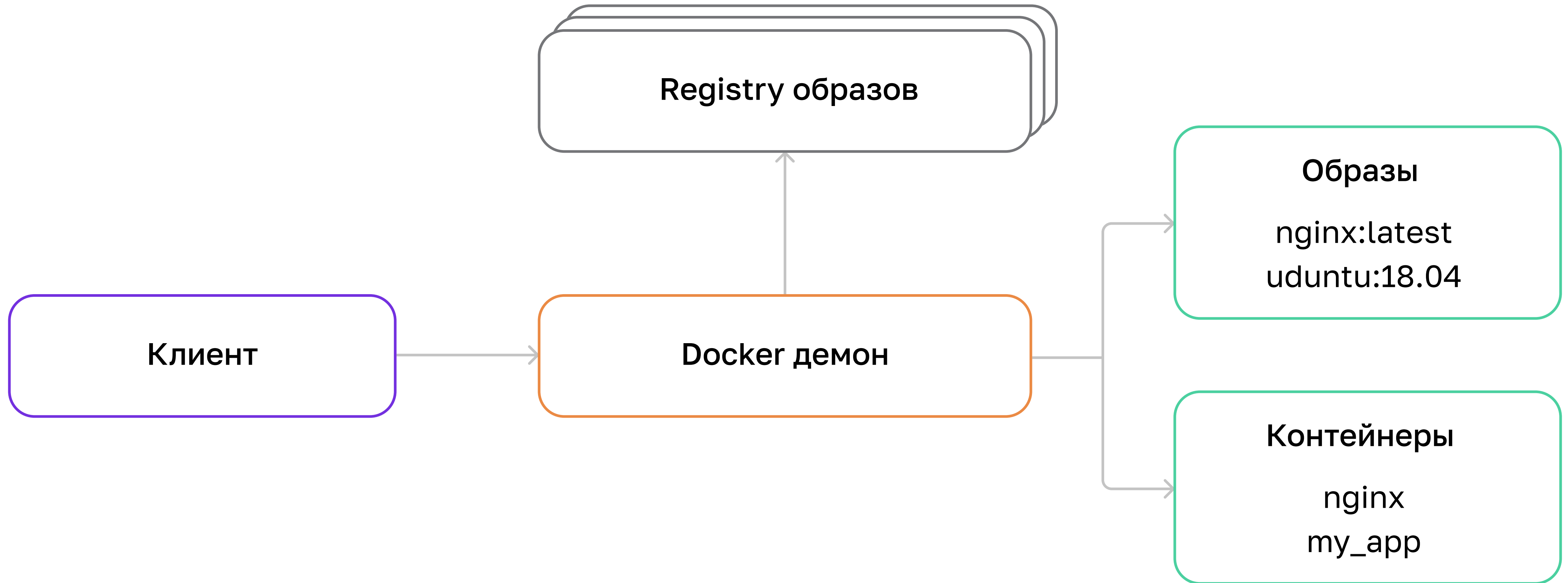


Преимущества Docker

- Декларативное описание упаковки вашего приложения вместе со всеми параметрами среды (Dockerfile)
- Приложение, среда и зависимости упаковываются вместе
- Образы разделены на слои, что позволяет переиспользовать их в случае отсутствия изменений
- Переносимость окружения, в котором работает приложение, позволяет его эксплуатировать, не боясь каких-либо особенностей среды, в которой будет запускаться контейнер

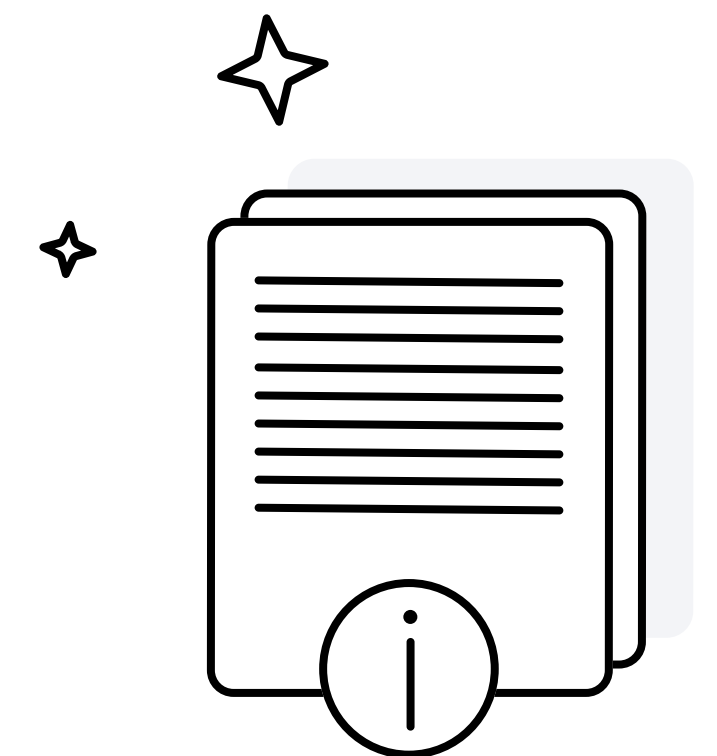


Архитектура Docker



Основные понятия

- **Контейнер** — это набор процессов, выполняемых в запущенной копии образа
- **Образ** — это некий шаблон, который используется для создания одинаковых контейнеров
- Образ состоит из определённой логической последовательности **слоёв**



Dockerfile

Dockerfile – простой текстовый файл, описывающий метаданные и слои будущего образа:

○ ○ ○

```
FROM openjdk:8-jdk-alpine
VOLUME /tmp
EXPOSE 8080
ADD build/libs/docker-spring-boot-1.0.jar springbootapp.jar
ENTRYPOINT ["java","-Djava.security.egd=file:/dev/./urandom","-jar","/springbootapp.jar"]
```

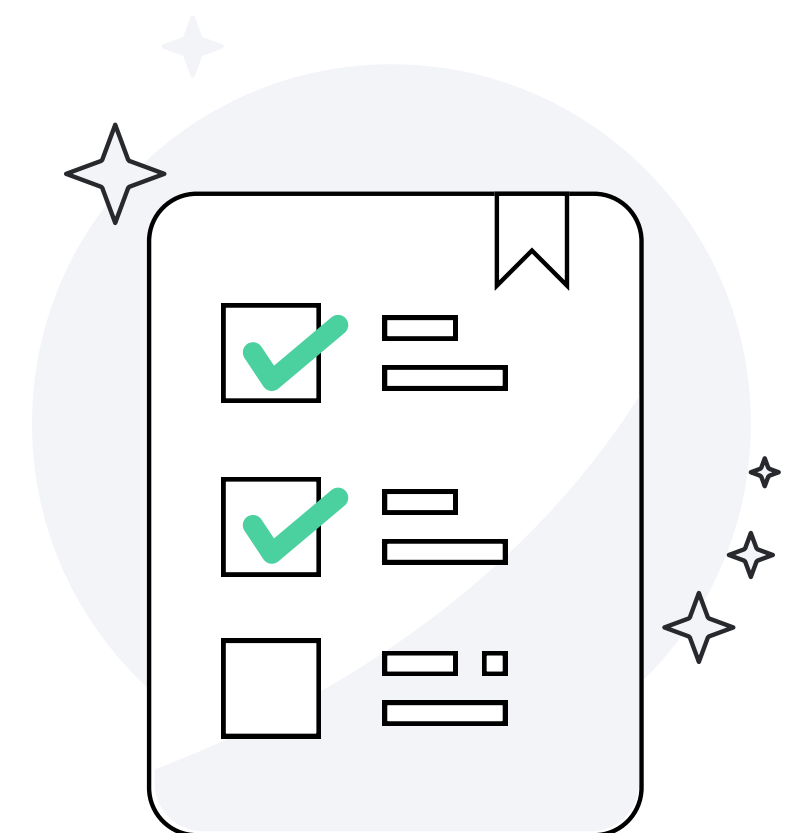
Инструкция FROM

FROM указывает на базовый образ для нашей сборки.

Базовый образ — это то, что является исходным слоем (или слоями) создаваемого образа. Базовый образ ещё называют родительским образом. Слои в итоговом образе создают только инструкции FROM, RUN, COPY и ADD.

Пример:

- FROM ubuntu



Инструкция RUN

RUN используется:

- для выполнения команды
- для записи изменений, внесённых в файловую систему
- для установки библиотек зависимостей, пакетов и различных файлов

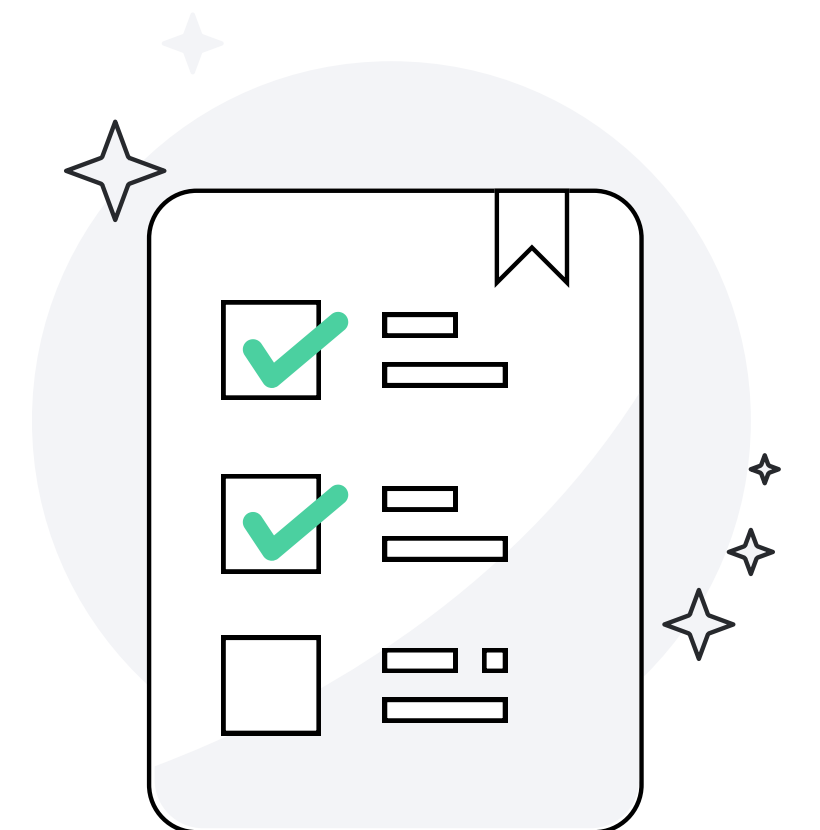


Инструкция COPY

Инструкция **COPY** добавляет в образ файлы и данные с вашего ПК:

`COPY . /src`

Это добавит содержимое контекста сборки (каталог, переданный в качестве аргумента для сборки docker) в каталог `/src` в контейнере



Инструкция ADD

ADD может получать файлы с удалённого хоста:

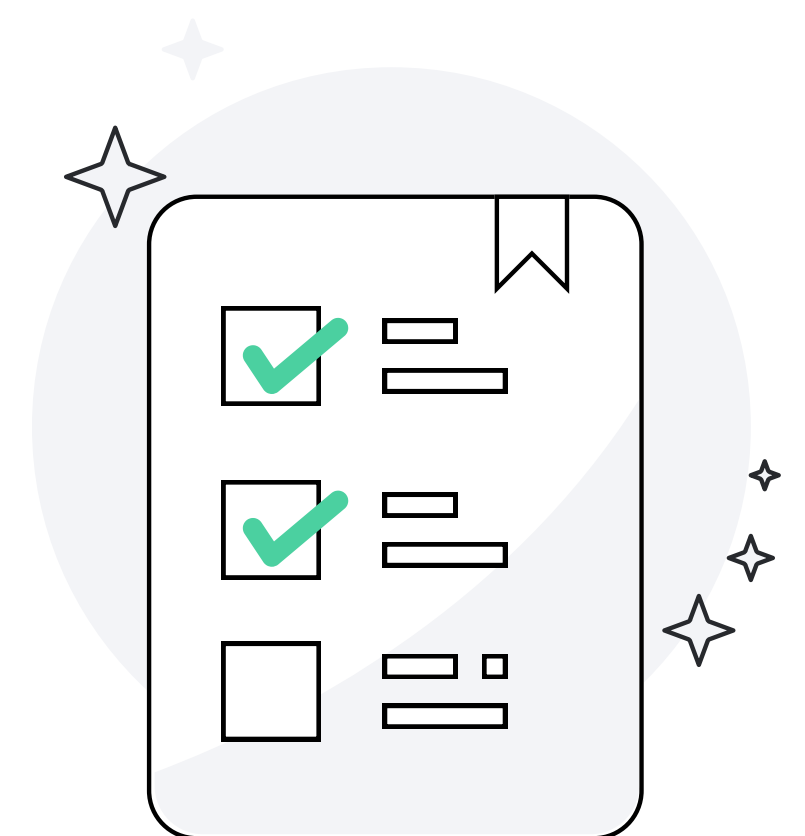
```
ADD http://www.example.com/webapp.jar / opt /
```

Это загрузит файл webapp.jar и поместит его в каталог / opt

ADD автоматически распакует zip-файлы и tar-архивы:

```
ADD ./assets.zip /var/www/htdocs/assets/
```

Это распакует assets.zip в /var/www/htdocs/assets



Инструкция ENV

Инструкция **ENV** определяет переменные окружения, которые должны быть установлены в любом контейнере, запущенном из образа:

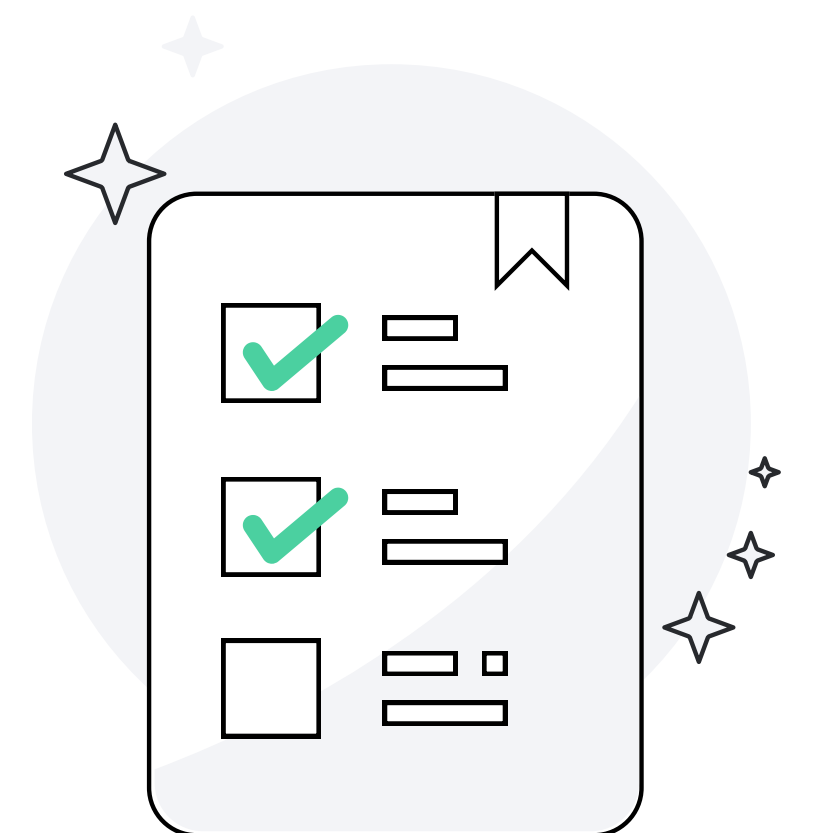
```
ENV WEBAPP_PORT 8080
```

Это приведёт к созданию переменной окружения в любых контейнерах, созданных из этого образа:

```
WEBAPP_PORT = 8080
```

Вы также можете указать переменные окружения при использовании **docker run**:

```
docker run -e WEBAPP_PORT = 8000 -e WEBAPP_HOST =  
www.example.com
```



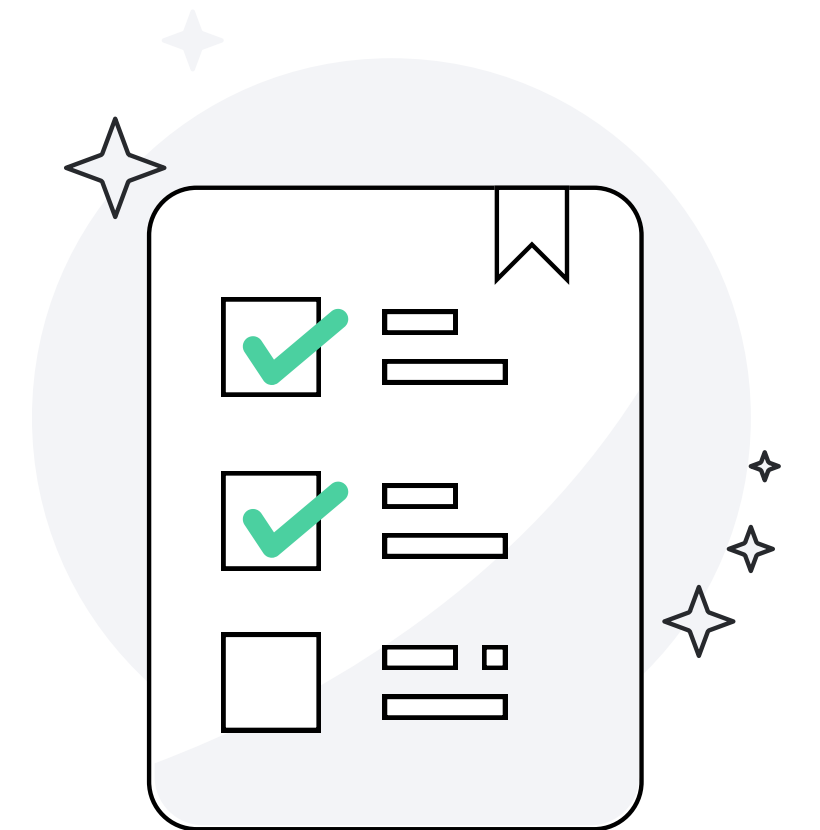
Инструкция EXPOSE

Инструкция **EXPOSE** сообщает Docker, какие порты должны быть открыты в этом образе:

EXPOSE 8080

EXPOSE 80 443

EXPOSE 53 / tcp 53 / udp



CMD и ENTRYPOINT

Команды **CMD** и **ENTRYPOINT** лучше всего работают вместе:

```
ENTRYPOINT ["nginx"]
```

```
CMD ["-g", "daemon off;"]
```

Параметры могут быть переопределены в командной строке:

```
docker run -d <dockerhubUsername> / image -binaryName
```

Дополнительные инструкции в Dockerfile

- **LABEL** добавляет к образу метаданные
- **ARG** определяет переменные окружения на время сборки
- **HEALTHCHECK** определяет команду, оценивающую состояние контейнера

Дополнительные инструкции в Dockerfile

SHELL устанавливает программу по умолчанию, которая будет использоваться для интерпретации строкового синтаксиса RUN, CMD

- WORKDIR — задаёт рабочую директорию для следующей инструкции
- user — устанавливает имя пользователя или UID для использования сборке из образа и при запуске контейнера
- VOLUME — создаёт точку монтирования для работы с постоянным хранилищем
- и т. д.

Best practice

- Используйте минимальное количество слоёв
- Указывайте сначала статичные и редко изменяемые инструкции

Best practice

- Используйте минимальное количество слоёв
- Указывайте сначала статичные и редко изменяемые инструкции
- Используйте кеш сборки для прироста скорости сборки